Appendix of "TempFlash: GPU-Resident, Single-Launch Training of Temporal Graph Neural Networks"

Weiwei Ye
The University of Tokyo
Japan
wwye@g.ecc.u-tokyo.ac.jp

Yankai Chen
MBZUAI
United Arab Emirates
yankaichan3@gmail.com

Zhen Wang
Huzhou Normal University
China
zhenwangrs@gmail.com

Dongyuan Li
The University of Tokyo
Japan
lidy@csis.u-tokyo.ac.jp

This document is the appendix of the paper. Sections 1–8, Tables 1 and 2, Figures 1–8 and Equations (1)–(7) refer to the paper; the works cited here are listed at the end of this appendix.

A PROOF OF PROPOSITION 1

Reference semantics. We restate the recent sampler of TGL with one history and one layer, as in our configuration. For node v with T-CSR segment $[o_v, o_{v+1})$, whose timestamps are non-decreasing, the pointer p_v is set to o_v at every reset. For a batch with roots (v_r, t_r), $r = 1, \ldots, R$, and fp32 thresholds $\theta_r = \mathrm{fl}(t_r - 10^{-7})$, the sampler first updates the pointers of all roots and then samples all roots. The update for root r scans $j = p_{v_r}, p_{v_r} + 1, \ldots$ and stops at the first j with $ts_j > \theta_r$; it sets $p_{v_r} \leftarrow j - 1$ if $j \neq p_{v_r}$, leaves the pointer unchanged otherwise, and sets it to $o_{v_r+1} - 1$ if the scan reaches the end of the segment. Updates of the same node are serialized by a lock in an arbitrary order. Sampling root r visits $k = p_{v_r}, p_{v_r} - 1, \ldots$ down to, but excluding, $\max(o_{v_r}, p_{v_r} - K)$ and outputs entry k if $ts_k < \theta_r$, in the order in which the entries are visited, i.e., from the most recent one backward.

Lemma 1 (one update). *Let u_r be the first index of the segment of v_r whose timestamp exceeds θ_r, or o_{v_r+1} if there is none. One update for root r maps p_{v_r} to* $\max(p_{v_r}, u_r - 1)$. The entries with timestamps above θ_r form the suffix $[u_r, o_{v_r+1})$ of the sorted segment. If $u_r > p_{v_r}$, the scan stops at $j = u_r \neq p_{v_r}$, or reaches the end of the segment if $u_r = o_{v_r+1}$, and in both cases the pointer becomes $u_r - 1$. If $u_r \leq p_{v_r}$, the scan stops at its first entry and the pointer is unchanged; for an empty segment, nothing is scanned and $u_r - 1 < o_{v_r} = p_{v_r}$. In every case, the result is $\max(p_{v_r}, u_r - 1)$.

Lemma 2 (order independence). *After the updates of a batch,* $p_v = \max\left(p_v^{\mathrm{old}}, \max_{r:v_r=v}(u_r - 1)\right)$ *for every order of the updates.* By Lemma 1, every update applies a map $x \mapsto \max(x, c_r)$, and such maps commute and compose to $x \mapsto \max(x, \max_r c_r)$, which is exactly the batch-level pointer update of Eq. (4).

Lemma 3 (search). *The upper bound of the query $(v_r \ll 32) \mid \varphi(\theta_r)$ in the key array is u_r.* The map φ of Eq. (2) orders all non-NaN fp32 values as the floating-point comparison does, except that it places -0 below $+0$; timestamps shifted to start at zero and thresholds obtained by an fp32 subtraction never take the value -0. Hence, for an entry i of node v_r, κ_i exceeds the query if and only if $ts_i > \theta_r$, while entries of other nodes compare like their node identifiers. Since the key array is sorted, its first entry above the query is u_r.

Proof of Proposition 1. The GPU sampler computes θ_r with the same fp32 subtraction. By Lemmas 2 and 3, the scatter with a maximum reduction yields the reference pointers after the update phase of every batch, by induction over the batches since the last reset. For root r, the reference outputs the entries $p - j$ for $j = 0, \ldots, m - 1$ with $m = \min(p - o_{v_r}, K)$ and $p = p_{v_r}$ whose timestamps are smaller than θ_r, in this order. Slot j of the GPU sampler reads entry $p - j$ and is valid under exactly these two conditions, so the valid slots, read row by row, form the reference sequence. Neighbor identifiers, event identifiers and timestamps are read from the same entries, and time deltas are computed with the same fp32 subtraction. □

Table 3: Commit semantics: how often an execution keeps an update other than the last occurrence (CPU: count; GPU and Write: % of the duplicated nodes).

Data	Model	Seq.	Batches	Updates	Dup.	CPU	GPU	Write
LastFM	TGN	mails	768	1.81M	296.2K	0	55.9%	57.1%
LastFM	TGN	memories	768	1.81M	296.2K	0	57.9%	58.6%
LastFM	APAN	mails	768	11.70M	678.0K	0	80.8%	81.2%
Wiki-Talk	TGN	mails	2,589	10.97M	1.64M	0	66.3%	67.0%
Wiki-Talk	TGN	memories	2,589	10.97M	1.64M	0	57.4%	58.5%
Wiki-Talk	APAN	mails	2,589	65.34M	6.33M	0	77.9%	78.1%
Stack-Ovf.	TGN	mails	300	1.49M	243.1K	0	61.8%	62.3%
Stack-Ovf.	TGN	memories	300	1.49M	243.1K	0	63.8%	64.5%
GDELT	TGN	mails	300	1.58M	148.4K	0	79.7%	80.0%
GDELT	TGN	memories	300	1.58M	148.4K	0	80.1%	80.6%

B CORRECTNESS TESTS

Sampler. Besides the four graphs of Table 1 (§5.3), we compared the GPU sampler with TGL's C++ sampler on 2,000 random graphs whose events are drawn so that the corner cases of the reference rule occur often: few distinct timestamps, many ties, batches whose roots repeat, nodes without or with only a few earlier events and roots whose threshold equals a stored timestamp. Each graph is processed in two passes with a reset in between, with $K \in \{1, 2, 3, 10\}$ and with one or eight sampler threads. Over 41,404 batches with 1.12M roots and 963.6K sampled neighbors, which contain 397,571 entries whose timestamp equals the threshold, 38,789 batches with repeated roots, 248,918 excluded first entries, 16,752 empty and 431,282 short histories, the two samplers return 0 differing neighbors, event identifiers, timestamps or orders.

Commit semantics. For every training batch, we built the sequences of candidate updates exactly as the reference does (TGN: mails interleaved per event, memories of all sources and then of all destinations; APAN: mails of all roots and then those to the sampled neighbors of every root) and compared three ways of executing ETC's selection with the last occurrence: on the CPU, on the GPU, and as a plain indexed GPU write without selection. Per sequence, Table 3 counts the inspected training batches (on Stack-Overflow and GDELT, 300 strided over the epoch), the updates and the nodes that occur more than once in their batch, summed over batches. It shows that the selection keeps the last occurrence whenever it executes sequentially on the CPU, whereas on the GPU it keeps

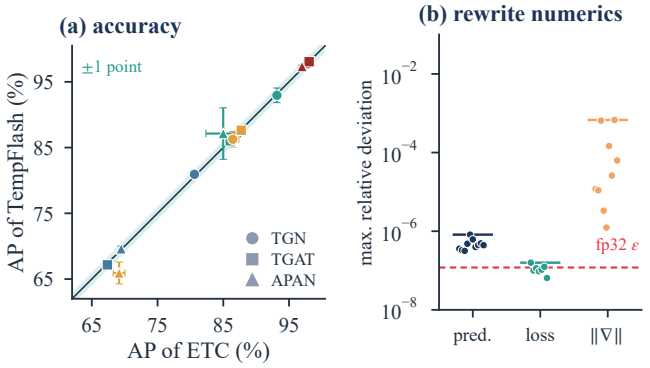


Figure 9: (a) Test AP of TempFlash versus ETC, both with the default configuration (three seeds; bars: one standard deviation; band: ±1 point). (b) Worst relative deviation between the rewritten and the reference operator order.

another update for 56–81% of the duplicated nodes, about as often as a plain indexed write without any selection. Two GPU executions of the same batch also disagree with each other. TempFlash instead commits the last occurrence of every node by construction (§3.5).

Differential test. TempFlash loads the initial parameters of an instance of ETC's model and trains side by side with ETC's training code on the same features, the same adaptive batches and the same negative destinations, with dropout disabled. After every batch, we compare predictions and losses (relative difference), all parameters after the Adam step, the memories and mails of every node touched by the batch (absolute difference), and the memory timestamps, mail timestamps and mailbox ring pointers (exact). The reference executes its selection on the CPU, so that it keeps the last occurrence (Table 3). Over all 768 training batches of a LastFM epoch, the discrete state of TGN and APAN agrees exactly (0 differing timestamps or pointers). Executed as released, i.e., with the selection on the GPU for TGN, the reference differs from TempFlash in 187,586 memory and 163,568 mail timestamps over the same 768 batches, and with the selection on the GPU for APAN in 242,221 and 764,260. With the state of TempFlash re-synchronized to the reference before every batch, the largest relative differences over 300 batches are 1.6×10^{-6} for predictions and 2.4×10^{-6} for parameters for all three models.

Rounding against fp64. On 6 synchronized TGN batches, an fp64 copy of the reference model evaluates the same sampled neighborhoods and pre-step states. The median relative prediction errors of the fp32 steps are 3.5×10^{-7} for TempFlash and 4.6×10^{-7} for the reference, and the largest relative gradient errors are 8.8×10^{-7} and 2.2×10^{-5}, i.e., TempFlash rounds no worse than the reference.

Rewrite. For every workload, we start the reference operator order and the project-once-gather-many order from identical parameters and states, train both for 100 to 150 steps and then compare predictions, losses, gradients and committed memories at ten further steps (Figure 9b). The worst relative deviations over all workloads are 8.2×10^{-7} for predictions, 1.6×10^{-7} for losses and 3.4×10^{-3} for gradient norms; the largest gradient deviations stem from ReLU inputs whose sign changes at rounding level. Across an AP range from 65% to 99%, the test AP of TempFlash with the default configuration lies close to that of ETC on the same workload (Figure 9a; mean absolute difference 0.59 points over the twelve workloads with three seeds of both systems).

Table 4: Epoch times (s) of our ports of Orca and Zebra to TGAT and APAN, and of TempFlash (Table 2).

	LastFM		Wiki-Talk		Stack-Overflow		GDELT	
	TGAT	APAN	TGAT	APAN	TGAT	APAN	TGAT	APAN
Orca	9.91	14.5	72.1	228.2	1390.7	2006.8	5095.3	1244.6
Zebra	36.6	41.6	212.4	418.5	2942.1	3208.6	9837.5	4900.3
TempFlash	1.64	3.49	13.2	15.7	126.0	159.2	330.8	347.3

Table 5: TempFlash with adaptive batches (adapt.) and SIMPLE's fixed batches (fixed), and SIMPLE: batches per epoch, epoch time (s) and test AP (%, mean over three seeds).

		Batches		Time (s)			AP (%)		
Data	Model	adapt.	fixed	adapt.	fixed	SIMPLE	adapt.	fixed	SIMPLE
LastFM	TGN	768	906	4.58	4.52	16.5	80.93	81.20	81.68
LastFM	TGAT	173	182	1.67	1.67	4.05	67.16	67.44	67.60
Wiki-Talk	TGN	2589	3656	32.2	32.1	100.6	92.95	92.94	84.14
Wiki-Talk	TGAT	847	1097	12.2	11.9	26.0	86.03	86.20	82.15
Stack-Ovf.	TGAT	6528	8890	126.0	125.7	384.3	87.64	87.37	87.70

C BASELINES

Orca and Zebra. Both extend the code base of TGN [1] and implement TGN only. We ported them to the two other models. TGAT is the memory-less special case of their architecture: our port removes the node memory and uses the node features as layer-0 representations; it keeps their two attention layers, Orca's embedding reuse and Zebra's PageRank-based neighbors. APAN is ported with the default APAN configuration: ten mails per node, the attention-based memory updater of TGL with layer normalization as the embedding, and delivery of every mail to the node and to its neighbors as sampled by the system (Orca: the ten most recent; Zebra: its top-k neighbors). Table 4 gives the epoch times of our ports, 3.6–29.7 times those of TempFlash.

SIMPLE's fixed batches. To separate SIMPLE's data placement from its batching, we also trained TempFlash with the default configuration and SIMPLE's fixed batches, timed next to SIMPLE (Table 5). With the same batches, TempFlash is 2.2–3.7× faster than SIMPLE with its default budget in the same session on the five workloads on which both ran. With SIMPLE's fixed batches, the test AP of TempFlash stays within the variation over seeds of its AP with adaptive batches on all five workloads with three fixed-batch seeds.

D ADDITIONAL DYNAMIC GRAPHS

Accuracy and speed. Table 6 lists the test AP over three seeds and the epoch times of TempFlash and ETC on the 36 workloads of §5.8. The test AP of the two systems differs by 0.69 points on average; the difference is at most twice the larger of their standard deviations over seeds, or one point, on 35 of them. Timed back to back with ETC on a dedicated GPU, TempFlash is 3.7–8.3× faster per epoch on all 36 of these workloads (geometric mean 5.2×).

Table 6: Additional graphs: seeds, test AP (%, mean ± standard deviation), epoch time (s) and speedup.

Data	Model	n	ETC AP	TempFlash AP	ETC	TempFlash	Spd.
Wikipedia	TGN	3	97.53 ± 0.27	97.30 ± 0.19	3.14	0.84	3.8×
	TGAT	3	88.64 ± 0.21	88.62 ± 0.25	2.46	0.37	6.6×
	APAN	3	91.07 ± 0.65	91.01 ± 0.27	2.87	0.50	5.7×
Reddit	TGN	3	99.14 ± 0.63	99.50 ± 0.05	18.1	3.72	4.9×
	TGAT	3	96.36 ± 0.25	96.09 ± 0.12	7.86	1.45	5.4×
	APAN	3	98.52 ± 0.60	98.75 ± 0.10	12.1	1.89	6.4×
MOOC	TGN	3	99.34 ± 0.04	99.37 ± 0.04	6.78	1.69	4.0×
	TGAT	3	97.13 ± 0.08	97.14 ± 0.02	5.17	0.72	7.1×
	APAN	3	98.87 ± 0.06	98.89 ± 0.09	5.39	0.98	5.5×
Enron	TGN	3	71.88 ± 2.02	70.84 ± 1.97	2.73	0.55	4.9×
	TGAT	3	70.75 ± 0.67	70.65 ± 0.40	1.95	0.31	6.3×
	APAN	3	52.99 ± 1.54	53.58 ± 1.96	1.85	0.33	5.6×
SocialEvo	TGN	3	80.56 ± 0.05	82.46 ± 0.31	38.1	7.69	5.0×
	TGAT	3	77.94 ± 0.07	77.90 ± 0.02	23.4	3.52	6.7×
	APAN	3	63.96 ± 4.86	60.77 ± 9.76	23.9	4.32	5.5×
UCI	TGN	3	80.15 ± 3.91	76.86 ± 7.22	1.29	0.33	3.9×
	TGAT	3	63.11 ± 1.25	63.17 ± 0.17	0.97	0.20	4.8×
	APAN	3	62.61 ± 2.17	62.47 ± 1.13	1.08	0.21	5.1×

Data	Model	n	ETC AP	TempFlash AP	ETC	TempFlash	Spd.
Flights	TGN	3	90.38 ± 1.63	90.85 ± 1.66	32.5	8.43	3.9×
	TGAT	3	82.12 ± 0.08	82.08 ± 0.06	19.6	3.33	5.9×
	APAN	3	87.35 ± 0.36	88.01 ± 1.46	24.0	4.43	5.4×
CanParl	TGN	3	72.09 ± 3.34	72.85 ± 1.44	3.06	0.37	8.3×
	TGAT	3	71.18 ± 0.22	71.26 ± 0.24	0.93	0.19	5.0×
	APAN	3	63.41 ± 0.67	64.76 ± 2.09	0.98	0.22	4.5×
USLegis	TGN	3	74.58 ± 0.68	75.88 ± 0.29	1.29	0.25	5.1×
	TGAT	3	68.08 ± 0.33	68.05 ± 0.59	0.85	0.13	6.4×
	APAN	3	65.66 ± 4.88	64.16 ± 3.98	0.81	0.16	5.1×
UNtrade	TGN	3	65.43 ± 0.59	65.58 ± 0.86	7.46	1.89	4.0×
	TGAT	3	53.37 ± 0.05	53.38 ± 0.04	5.63	0.87	6.5×
	APAN	3	60.01 ± 1.62	60.90 ± 3.32	5.90	1.09	5.4×
UNvote	TGN	3	58.56 ± 1.41	57.96 ± 0.35	14.1	3.84	3.7×
	TGAT	3	50.93 ± 0.01	50.95 ± 0.03	11.0	1.71	6.4×
	APAN	3	55.60 ± 1.27	55.28 ± 1.10	10.9	2.18	5.0×
Contacts	TGN	3	89.97 ± 0.17	89.56 ± 1.12	42.1	10.4	4.1×
	TGAT	3	88.45 ± 0.08	88.38 ± 0.10	28.0	4.59	6.1×
	APAN	3	87.05 ± 1.43	82.48 ± 5.29	24.7	5.83	4.2×

Table 7: Device memory (GB) allocated by TempFlash per workload, by array, and the edge features kept in the zero-copy host tier. ^{p}: bit-packed; ^{f}: factorized mailbox.

Data	Model	Entries	T-CSR	Events	Node feat.	Edge feat.	States	Other	Total	Host
LastFM	TGN	2.59M	0.05	0.01	0.00	0.62	0.00	0.12	0.80	0.0
LastFM	TGAT	2.59M	0.05	0.01	0.00	0.62	0.00	0.09	0.77	0.0
LastFM	APAN	2.59M	0.05	0.01	0.00	0.62	0.03	0.10	0.80	0.0
Wiki-Talk	TGN	7.83M	0.16	0.09	0.73	5.02	2.03	0.17	8.20	0.0
Wiki-Talk	TGAT	7.83M	0.16	0.09	0.73	5.02	0.00	0.13	6.13	0.0
Wiki-Talk	APAN	7.83M	0.16	0.09	0.00	5.02	9.07^{f}	0.13	14.47	0.0
Stack-Ovf.	TGN	63.50M	1.22	0.71	1.67	0.00	4.63	0.27	8.50	40.7
Stack-Ovf.	TGAT	63.50M	1.22	0.71	1.67	0.00	0.00	0.21	3.81	40.7
Stack-Ovf.	APAN	63.50M	1.22	0.71	0.00	0.00	20.70^{f}	0.22	22.85	40.7
GDELT	TGN	191.29M	3.56	2.14	0.03	4.10^{p}	0.03	0.39	10.24	0.0
GDELT	TGAT	191.29M	3.56	2.14	0.03	4.10^{p}	0.00	0.37	10.19	0.0
GDELT	APAN	191.29M	3.56	2.14	0.03	4.10^{p}	0.24	0.35	10.42	0.0

Algorithm 1 Captured training step for bucket B

Require: cursor c; batch starts $lo[\cdot]$ and sizes $sz[\cdot]$; event arrays; T-CSR; tiers; states

1: $n \leftarrow sz[c]$; $ok \leftarrow [0, B) < n$; $ev \leftarrow \min(lo[c] + [0, B),\ |E| - 1)$ ▷ device cursor
2: $(\mathbf{src}, \mathbf{dst}, \mathbf{t}) \leftarrow$ events$[ev]$; $\mathbf{neg} \leftarrow$ randint$(|V|, B)$
3: roots $\leftarrow [\mathbf{src}; \mathbf{dst}; \mathbf{neg}]$ at times $[\mathbf{t}; \mathbf{t}; \mathbf{t}]$
4: $(nbr, eid, \Delta t, mask) \leftarrow$ Sample(roots, ok) ▷ §3.2
5: gather features, memories and mails in slot layout ▷ §3.4, §3.6
6: $\mathcal{L} \leftarrow$ MaskedBCE(Model$(\cdot)$, ok); $\nabla\mathcal{L}$; Adam step ▷ §3.3
7: Commit(new memories and mails, ok) ▷ §3.5
8: $c \leftarrow c + 1$

Table 8: Time (s) to sample the training batches of one epoch with TGL's C++ sampler and with the GPU sampler of TempFlash. Nbrs.: sampled neighbors per epoch.

		TGL sampler (threads)			TempFlash sampler	
Data	Nbrs.	1	8	32	eager	captured
LastFM	16.2M	1.5	1.2	1.2	0.58	0.12
Wiki-Talk	57.6M	10.3	5.4	5.7	2.08	0.50
Stack-Ovf.	673.6M	99.1	49.7	49.6	11.64	3.25
GDELT	1586.6M	165.3	119.9	198.8	30.41	6.93

E IMPLEMENTATION DETAILS

Sampling time. Table 8 times the sampling of the training batches of one epoch with TGL's C++ sampler on 1, 8 and 32 host threads, and with the GPU sampler of TempFlash, launched kernel by kernel (eager) or replayed from a captured graph. Both samplers process the same roots and return the same neighbors, whose total number per epoch is given in column Nbrs. The captured GPU sampler is 9.9–17× faster than TGL's sampler with eight threads and 10–29× faster than with 32 threads.

Memory. Table 7 splits the device memory of TempFlash into the T-CSR (Entries: its number of entries; T-CSR: its size including the sorting keys), the event arrays, node and edge features, node states (memories and mailboxes) and other tensors, including the workspace of a step; Host gives the edge features kept in the zero-copy host tier, i.e., in pinned host memory.

Step anatomy. Table 9 dissects one training step with all data on the GPU: it launches 344–485 kernels, and replaying them from a captured graph removes the dispatch of every kernel by the host and makes the step up to 1.9× faster, most for memory-based models with small batches.

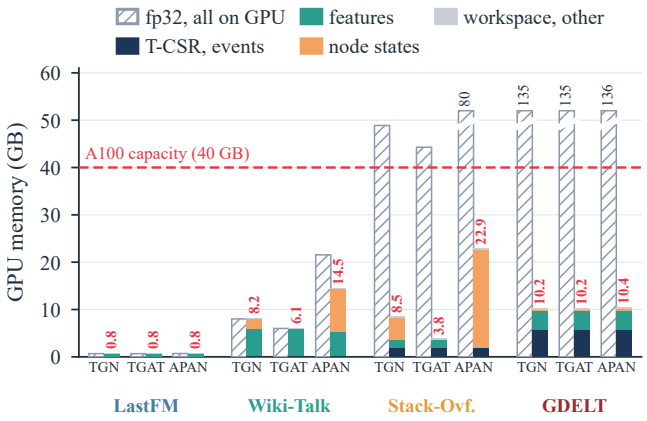


Figure 10: GPU memory per workload with all data in fp32 (hatched) and as allocated by TempFlash (solid).

Table 9: Kernels and wall time per training step with the default configuration and all data on the GPU, launched one by one (Eager) or replayed from a captured graph (Replay).

Data	Model	Kernels	Eager (ms)	Replay (ms)	Ratio
LastFM	TGN	474	8.0	6.0	1.3×
	TGAT	349	10.5	9.7	1.1×
	APAN	377	6.6	3.4	1.9×
Wiki-Talk	TGN	468	13.0	12.4	1.0×
	TGAT	345	15.0	14.5	1.0×
	APAN	392	6.7	6.1	1.1×
Stack-Ovf.	TGN	470	17.9	17.3	1.0×
	TGAT	344	19.1	18.6	1.0×
	APAN	393	10.0	9.4	1.1×
GDELT	TGN	485	13.4	12.7	1.1×
	TGAT	350	11.5	11.0	1.0×
	APAN	379	7.6	7.0	1.1×

REFERENCES

[1] Emanuele Rossi, Ben Chamberlain, Fabrizio Frasca, Davide Eynard, Federico Monti, and Michael Bronstein. 2020. Temporal Graph Networks for Deep Learning on Dynamic Graphs. *arXiv preprint arXiv:2006.10637* (2020).